

Практика 3. Типы в λ -исчислении

осень 2026

Андрей Стоян, Илья Колегов

осень 2026

Летучка 3

Летучка 3 · 7 минут · Ваше ФИ: _____

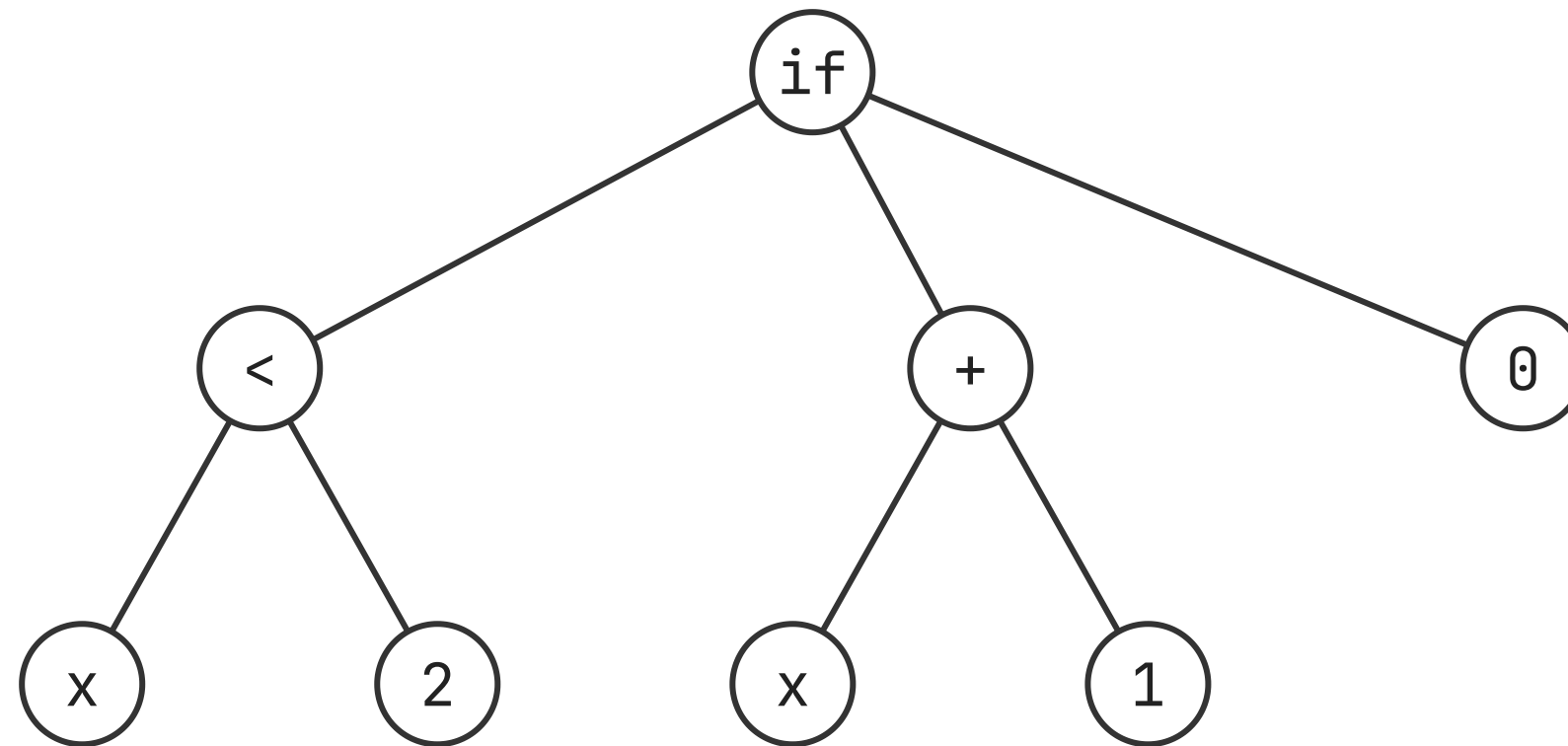
1. Расставьте все скобки: $f\ x\ y\ z \equiv$ _____
2. Какой λ связывает последнее вхождение x в терме $\lambda_1 x. \lambda_2 y. x (\lambda_3 x. y\ x)$? Номер: ____
3. Сделайте один β -шаг: $(\lambda f\ x. f\ (f\ x))\ \mathbf{not} \rightsquigarrow$ _____
4. Как называются функции, принимающие функции в качестве аргументов? _____
5. Свойство комбинатора неподвижной точки: $\mathbf{Y}\ F =_{\beta}$ _____

Идея систем типов

Анализ типов: метки от листьев к корню

Программа — это дерево. Попробуем приписать каждой вершине метку-тип, двигаясь от листьев к корню.

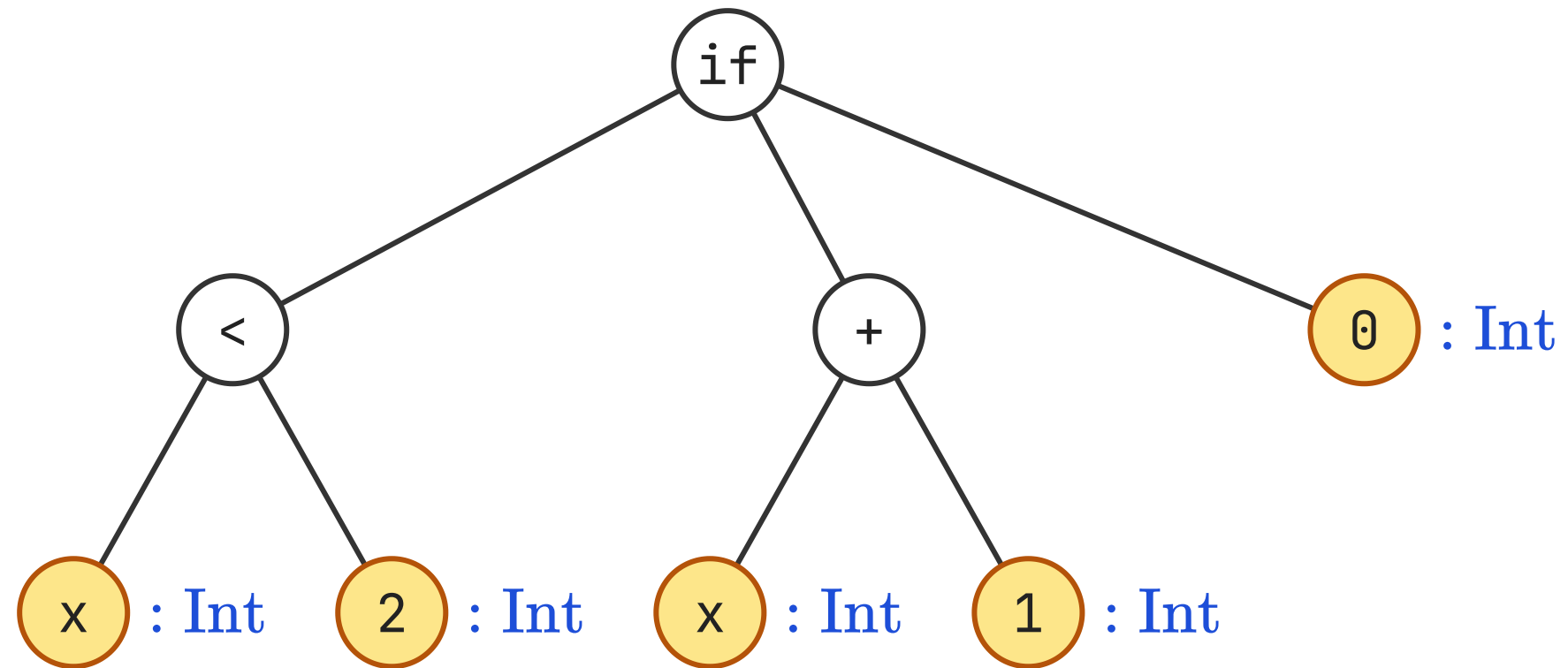
if (x < 2) (x + 1) 0



Анализ типов: метки от листьев к корню

Листья. Константа знает свою метку сама; про переменную x нужно знать заранее — пусть $x : \text{Int}$.

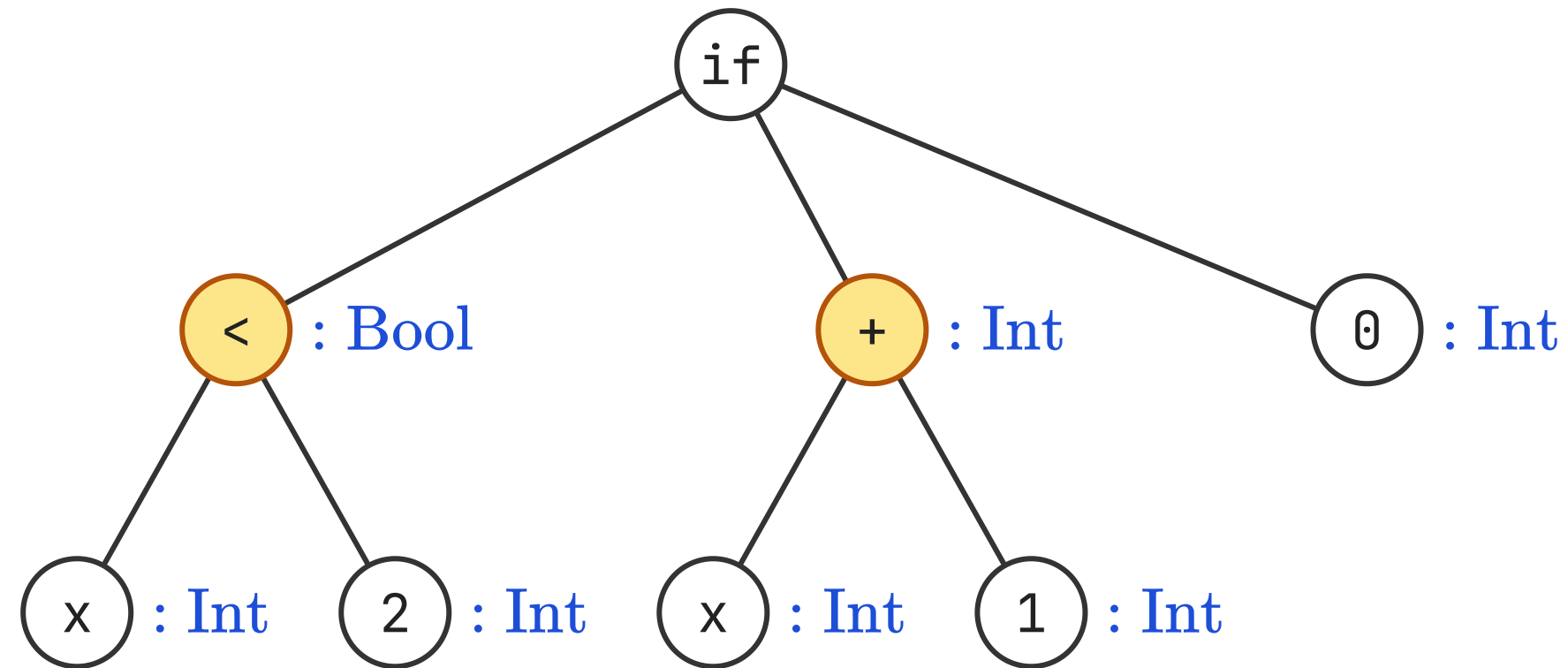
if ($x < 2$) ($x + 1$) 0



Анализ типов: метки от листьев к корню

Правило для $<$: если оба поддерева помечены `Int`, вершина получает метку `Bool`. Правило для $+$: оба поддерева `Int` — метка `Int`.

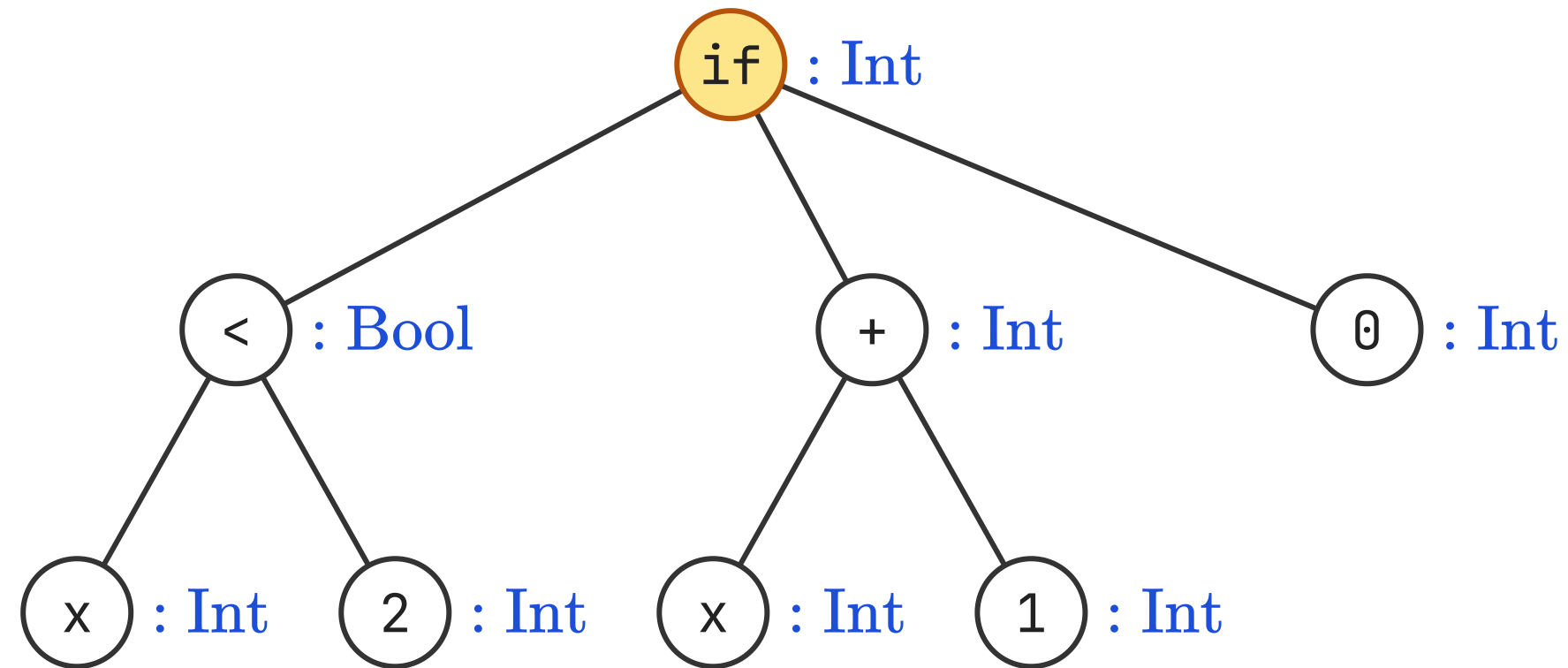
`if (x < 2) (x + 1) 0`



Анализ типов: метки от листьев к корню

Правило для if: условие помечено Bool, ветки помечены одинаково — вершина получает метку веток.
Помечено всё дерево: программа «хорошая».

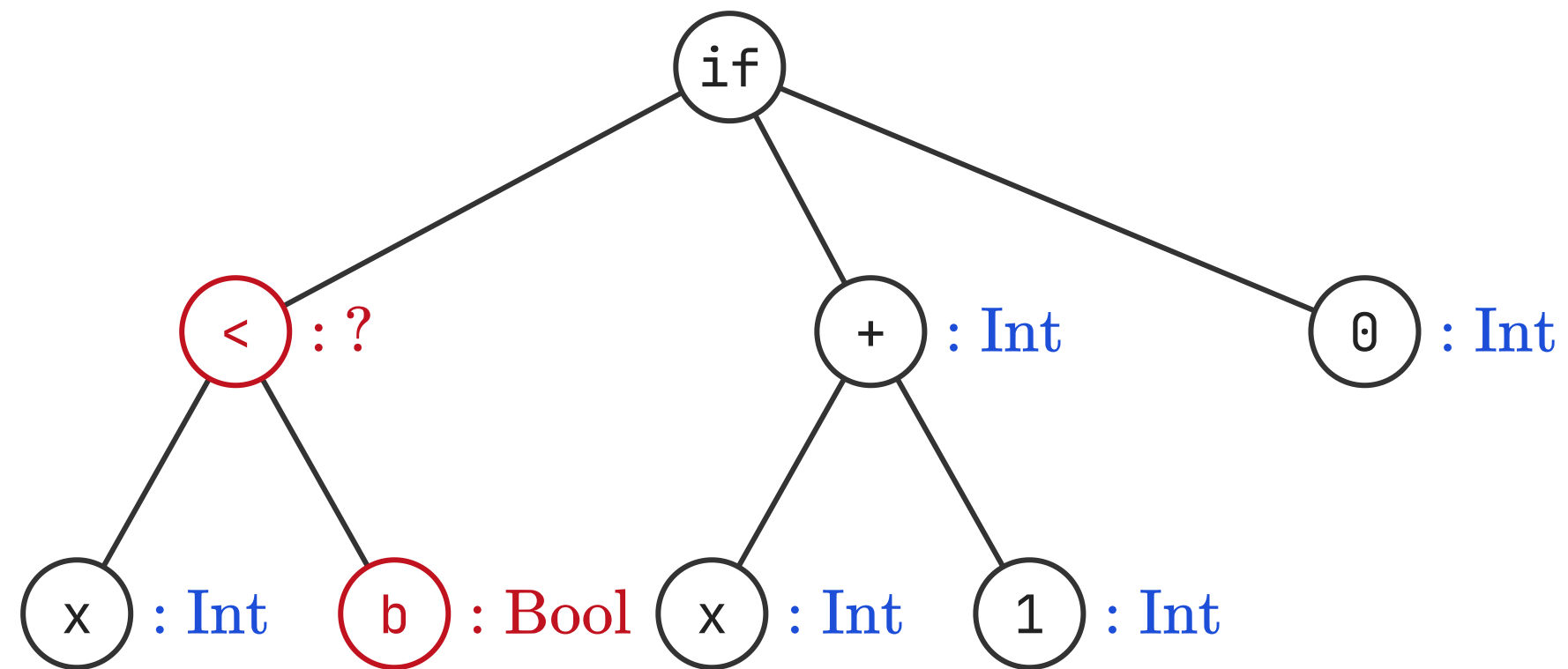
if (x < 2) (x + 1) 0



Анализ типов: метки от листьев к корню

А если сравнить x с переменной b , про которую дано $b : \text{Bool}$? Правило для $<$ не срабатывает, метки нет, выше идти не с чем: программа отвергнута до запуска.

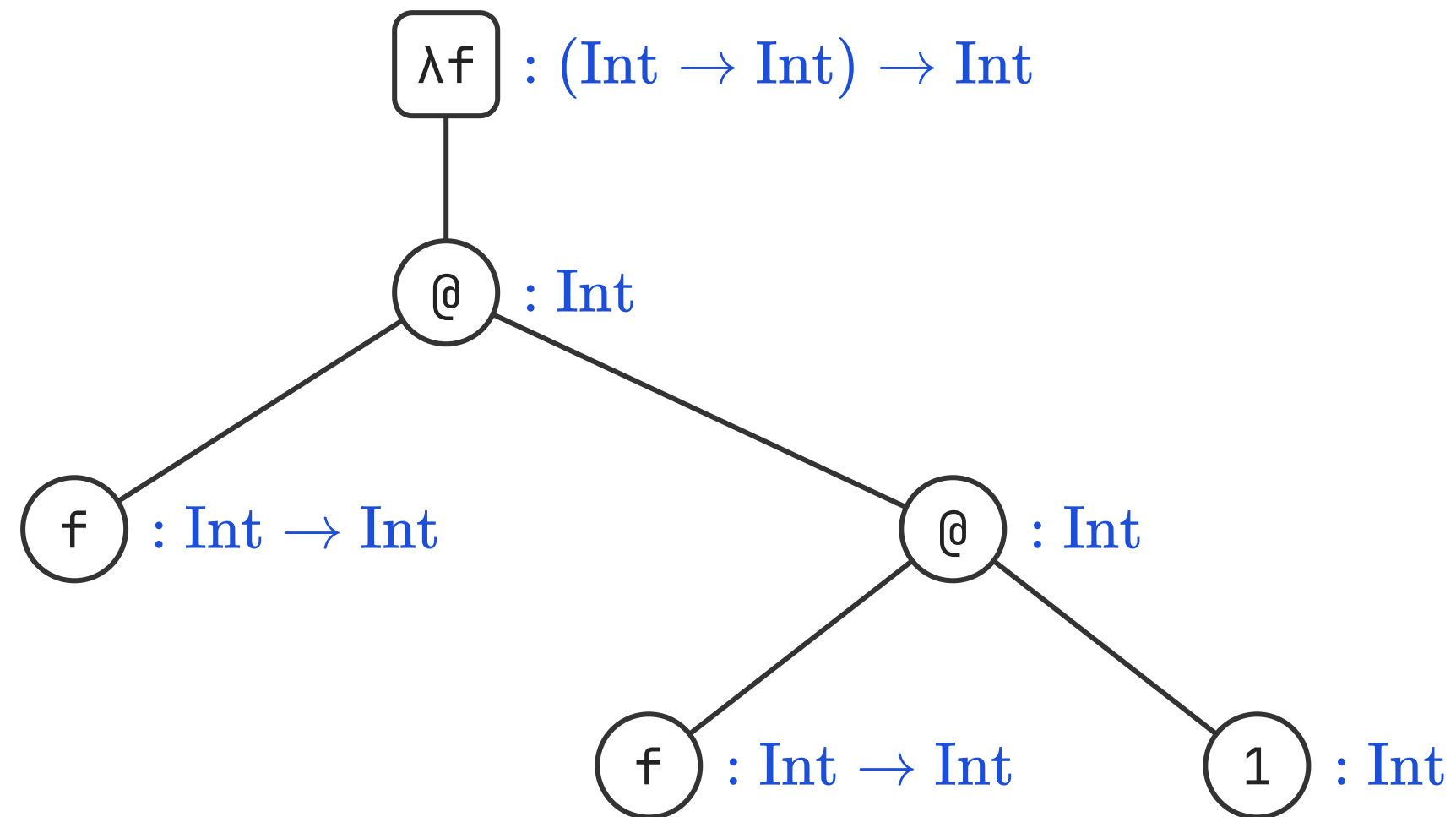
if ($x < b$) ($x + 1$) 0



Типы — тоже деревья

Здесь f помечена $\text{Int} \rightarrow \text{Int}$, а вся программа — $(\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int}$.

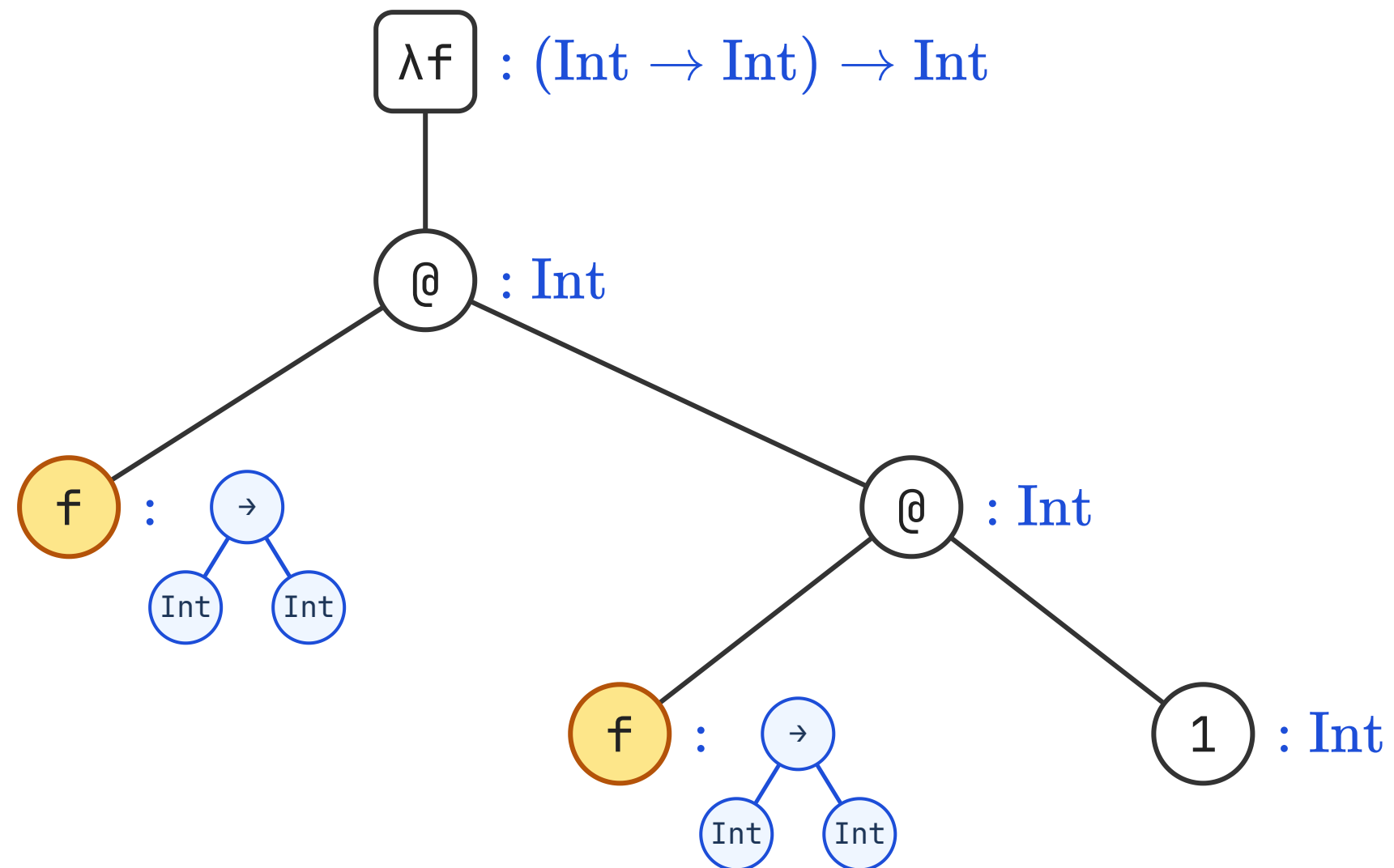
$\lambda f. f (f 1)$



Типы — тоже деревья

Стрелка — узел с двумя поддеревьями: слева тип аргумента, справа тип результата. Листья — константы `Int`, `Bool` и типовые переменные α , β .

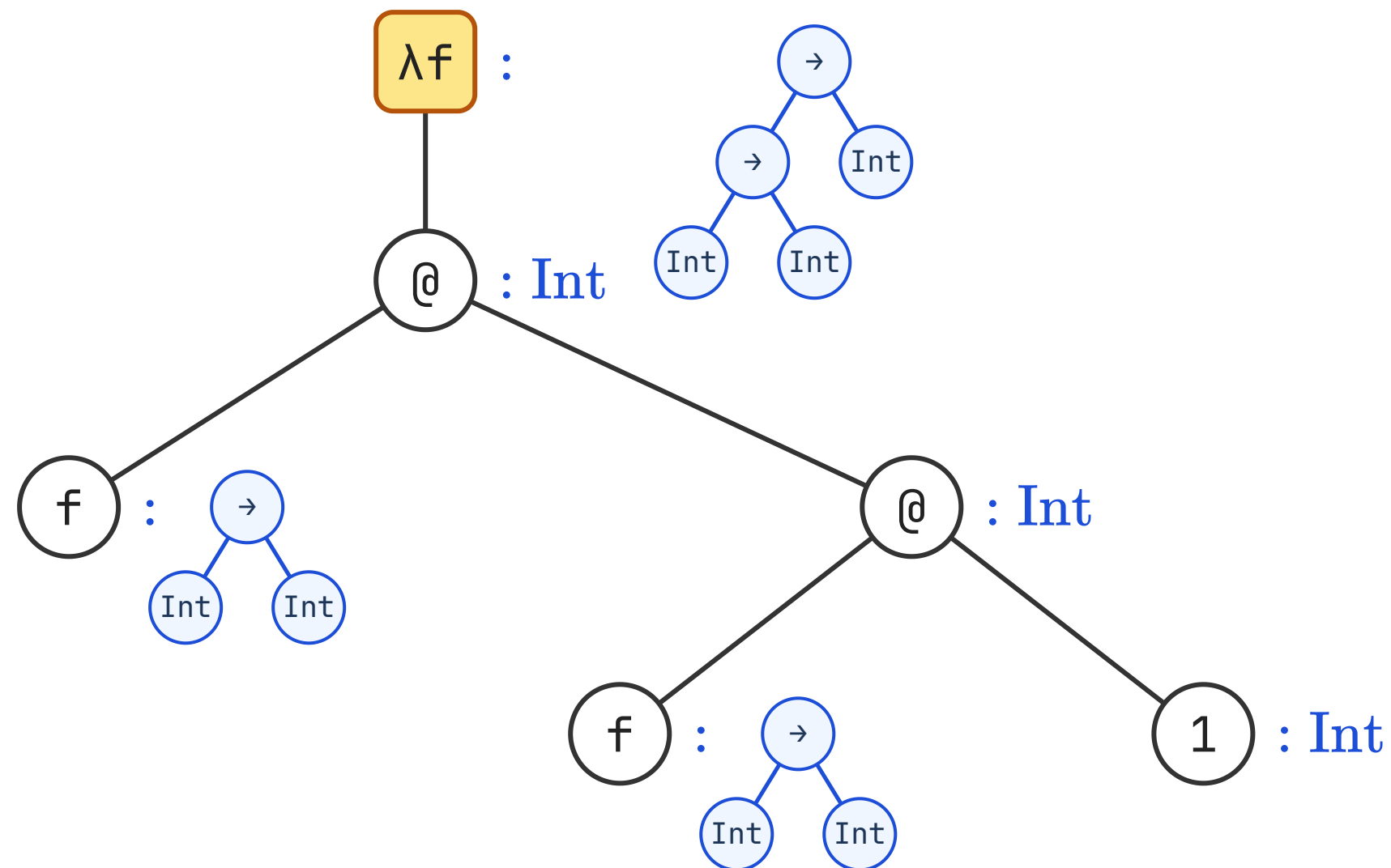
$\lambda f. f (f 1)$



Типы — тоже деревья

Тип всей программы — тоже дерево: $(\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int}$. Скобки в записи — это форма дерева: слева поддерево $\text{Int} \rightarrow \text{Int}$, программа принимает функцию.

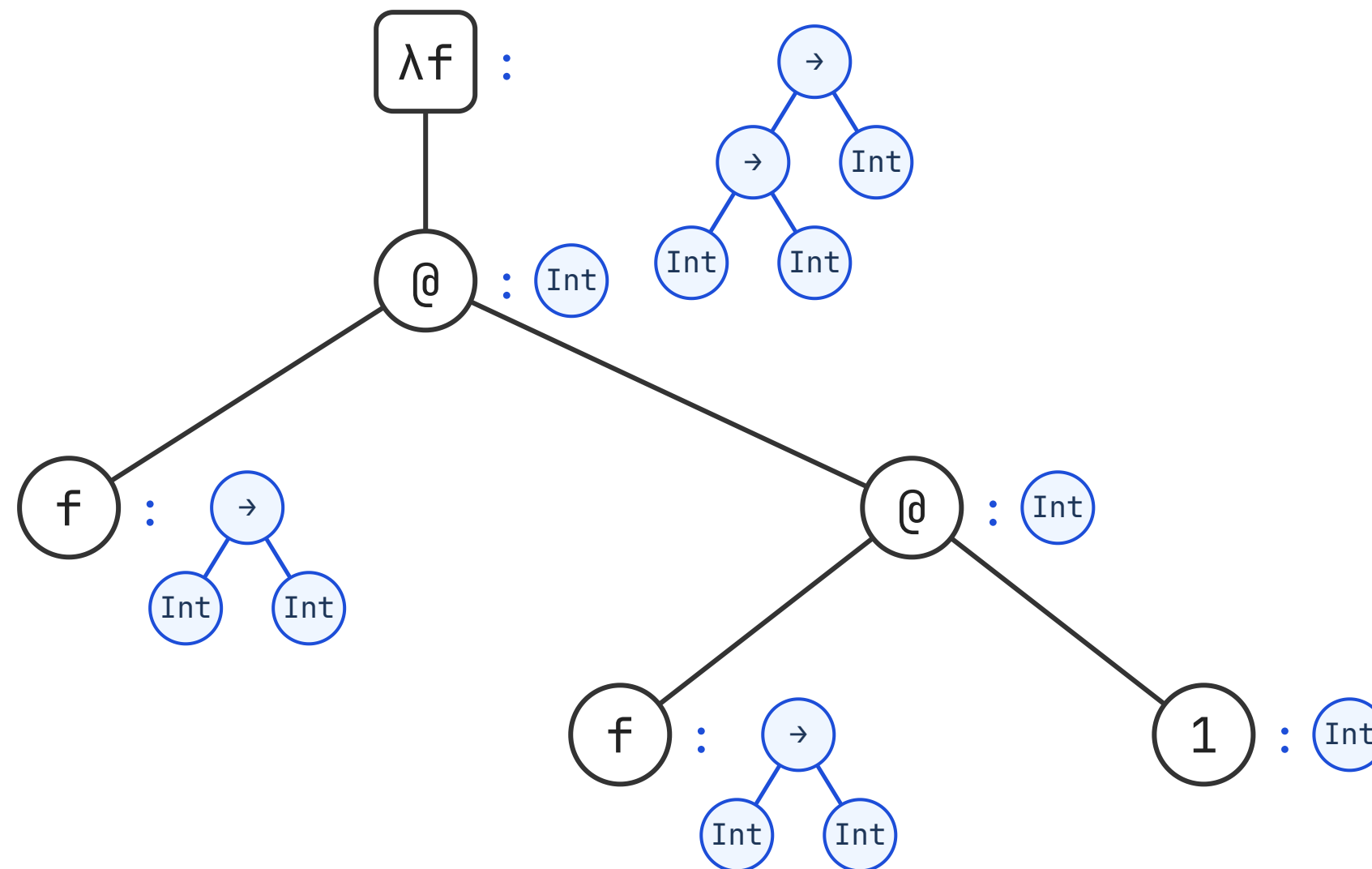
$\lambda f. f (f 1)$



Типы — тоже деревья

Остальные метки — деревья из одного листа. Итог: анализ типов аннотирует дерево программы деревьями типов; осталось договориться, по каким правилам метка вершины получается из меток поддеревьев.

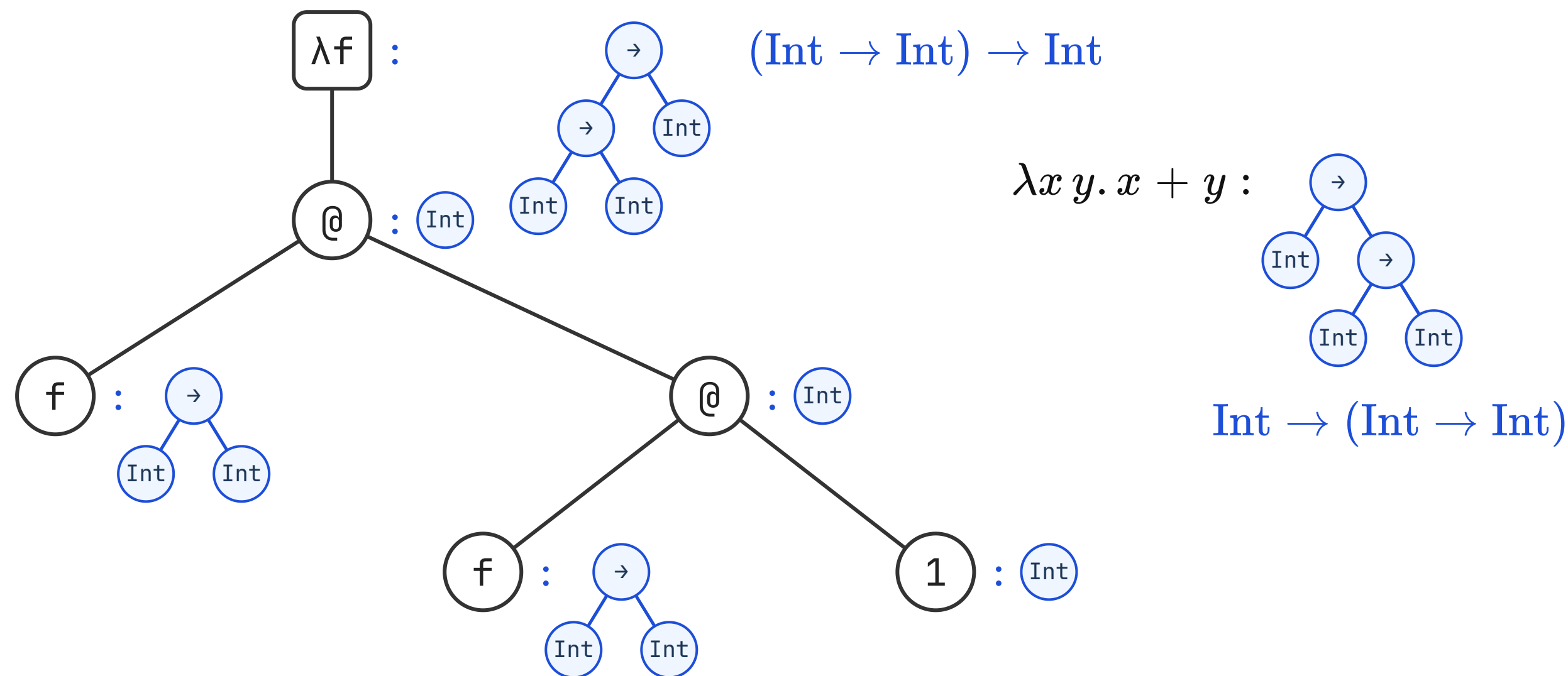
$\lambda f. f (f 1)$



Типы — тоже деревья

Без скобок стрелка группируется вправо: $\lambda x y. x + y : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$ — это $\text{Int} \rightarrow (\text{Int} \rightarrow \text{Int})$, функция, возвращающая функцию. У нашей программы скобки слева — другое дерево.

$\lambda f. f (f 1)$



Анализ типов

- Попробуем присписать каждой ноде программы синтаксические метки по определённым правилам
- Если метки таким образом присписать можно — программа «хорошая»
- Вид таких меток и правил задаётся *системой типов*
- Анализ типов — разновидность статического анализа: программу проверяем, не запуская

$\lambda_{\rightarrow}$ ИНТУИТИВНО

Наш язык на сегодня: $\lambda_{\rightarrow}$ и константы

Синтаксис $\lambda_{\rightarrow}$ — как в лекции + наше расширение

- Типы: $T ::= \alpha \mid T \rightarrow T \mid \text{Int} \mid \text{Bool}$
- Термы: $M ::= x \mid M M \mid \lambda x. M$ по Чёрчу — $\lambda x^T. M$
 - | $n \mid M + M \mid M - M \mid M * M$, где $n \in \mathbb{N} = \{0, 1, 2, \dots\}$
 - | $b \mid M < M \mid M == M \mid \text{if } M M M$, где $b \in \mathbb{B} = \{\text{true}, \text{false}\}$

Расширение — числа и були встроены в язык как новые формы терма. Это не **plus** и **true** из практики
1: раскрыть их в λ -терм нельзя; $2 + 3 \rightsquigarrow_+ 5$ происходит за один шаг

Типизация интуитивно: конкретные типы

✎ Припишите тип терму $\lambda x. x + 1$

⚡ **Решение.**

Функция: $? \rightarrow ?$. Складывать можно только числа — $x : \text{Int}$, сумма — число. Ответ: $\text{Int} \rightarrow \text{Int}$

✎ Припишите тип терму $\lambda b. \text{if } b \ 0 \ 1$

⚡ **Решение.**

b — условие: $b : \text{Bool}$; обе ветки Int , значит и всё **if** — Int . Ответ: $\text{Bool} \rightarrow \text{Int}$

✎ Припишите тип терму $\text{true} + 1$

⚡ **Решение.**

Типа нет: слагаемые обязаны быть числами, а $\text{true} : \text{Bool}$

Типизация интуитивно: буквы

✎ Найдите наиболее общий тип терма $\lambda x y. x$

✂ **Решение.**

Типы аргументов в терме не записаны, а тело их никак не ограничивает — выбираем различные буквы: $\alpha \rightarrow \beta \rightarrow ?$. Результат — первый аргумент. Ответ: $\alpha \rightarrow \beta \rightarrow \alpha$

✎ Найдите наиболее общий тип терма $\lambda x y f. f x y$

✂ **Решение.**

f в теле принимает два аргумента — x и y , на них ограничений нет: $\alpha \rightarrow \beta \rightarrow (\alpha \rightarrow \beta \rightarrow ?) \rightarrow ?$.
Результат f ничем не ограничен — берём свежую букву. Ответ: $\alpha \rightarrow \beta \rightarrow (\alpha \rightarrow \beta \rightarrow \gamma) \rightarrow \gamma$

Типизация интуитивно: функции-аргументы

✎ Припишите тип терму $\lambda x^\alpha f^{\alpha \rightarrow \beta}. f\ x$ — здесь типы аргументов уже подписаны

✂ **Решение.**

Видим абстракцию — пишем стрелку $\alpha \rightarrow ?$ и типизируем тело, зная x^α . Тело — тоже функция: $\alpha \rightarrow ((\alpha \rightarrow \beta) \rightarrow ?)$. Внутри знаем x^α и $f^{\alpha \rightarrow \beta}$: тип x совпадает с типом аргумента f , значит $f\ x : \beta$. Ответ: $\alpha \rightarrow (\alpha \rightarrow \beta) \rightarrow \beta$

✎ Найдите наиболее общий тип терма $\lambda f\ x. f\ (f\ x)$

✂ **Решение.**

f применяют, значит $f : \alpha \rightarrow \beta$. Результат $f\ x$ снова отдадут в f , значит β и α — один тип; x — того же типа. Ответ: $(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$

Типизация самоприменения

 **Устно:** А какой тип у $\lambda x. x\ x$?

 **Решение.**

Никакого. x применяют — значит $x : \alpha \rightarrow \beta$; аргументом стоит тот же x — значит $x : \alpha$. Получается $\alpha \equiv \alpha \rightarrow \beta$: тип должен быть собственной частью, а у конечного дерева так не бывает.

Население типов интуитивно

✎ Населите тип $(\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma$ — то есть постройте терм такого типа

✚ Решение.

- Вспоминаем про ассоциативность стрелки: $(\beta \rightarrow \gamma) \rightarrow ((\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma)$
- Это явно функция, пишем абстрактор: $\lambda f^{\beta \rightarrow \gamma}. \underbrace{\square}_{(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \gamma}$
- Аналогично ещё две стрелки дают два аргумента: $\lambda f^{\beta \rightarrow \gamma} g^{\alpha \rightarrow \beta} x^{\alpha}. \underbrace{\square}_{\gamma}$
- Единственный способ получить значение типа γ — вызвать f : $\lambda f^{\beta \rightarrow \gamma} g^{\alpha \rightarrow \beta} x^{\alpha}. f \underbrace{\square}_{\beta}$
- Конструируем подходящий аргумент для f типа β вызовом g : $\lambda f^{\beta \rightarrow \gamma} g^{\alpha \rightarrow \beta} x^{\alpha}. f (g \underbrace{\square}_{\alpha})$
- Воспользуемся x , чтобы достроить терм нужного типа: $\lambda f^{\beta \rightarrow \gamma} g^{\alpha \rightarrow \beta} x^{\alpha}. f (g x)$

Буквы и константы

✎ Устно:

- (а) $\lambda x. x : \alpha \rightarrow \alpha$. Какие из типов тоже подходят этому терму: $\text{Int} \rightarrow \text{Int}$, $(\beta \rightarrow \gamma) \rightarrow (\beta \rightarrow \gamma)$, $\text{Int} \rightarrow \text{Bool}$, $\alpha \rightarrow \beta$?
- (б) $\lambda x. x + 1 : \text{Int} \rightarrow \text{Int}$. А тип $\alpha \rightarrow \alpha$ ему подходит?
- (в) $\lambda x y. x$ имеет типы $\alpha \rightarrow \alpha \rightarrow \alpha$ и $\alpha \rightarrow \beta \rightarrow \alpha$. Который из них главнее?

✚ Решение.

- (а) Первые два. Вместо **буквы** подставляется любой тип, но один и тот же во всех её вхождениях: $\alpha \rightarrow \alpha$ — не «из любого в любой»
- (б) Нет. **Константный тип** Int конкретен, у него свои значения $0, 1, 2, \dots$: букву превратить в Int можно, Int в букву — нет
- (в) Второй: первый получается из него подстановкой $\beta := \alpha$, обратно — нет. **Наиболее общий тип** — тот, из которого все остальные получаются подстановкой вместо букв

$\lambda_{\rightarrow}$ формально

Контекст типизации

- В программах используются введённые ранее имена
- Для типизации нужно знать типы сущностей, связанных ранее с этими именами
- Будем поддерживать доп. структуру данных, которую будем заполнять именами с соответствующими им типами в процессе обхода дерева программы

Пример на Си

```
int dostrange(int a, char i) {  
    // point 0  
    int c = 5;  
    for (int i = 0; i < 15; ++i) {  
        i -= 1; // point 1  
        long r = i;  
        i *= r; // point 2  
    }  
    return a + (int)i * c; // point 3  
}
```

Контекст типизации в каждой из точек:

0. a^{int} , i^{char}

1. a^{int} , c^{int} , i^{int}

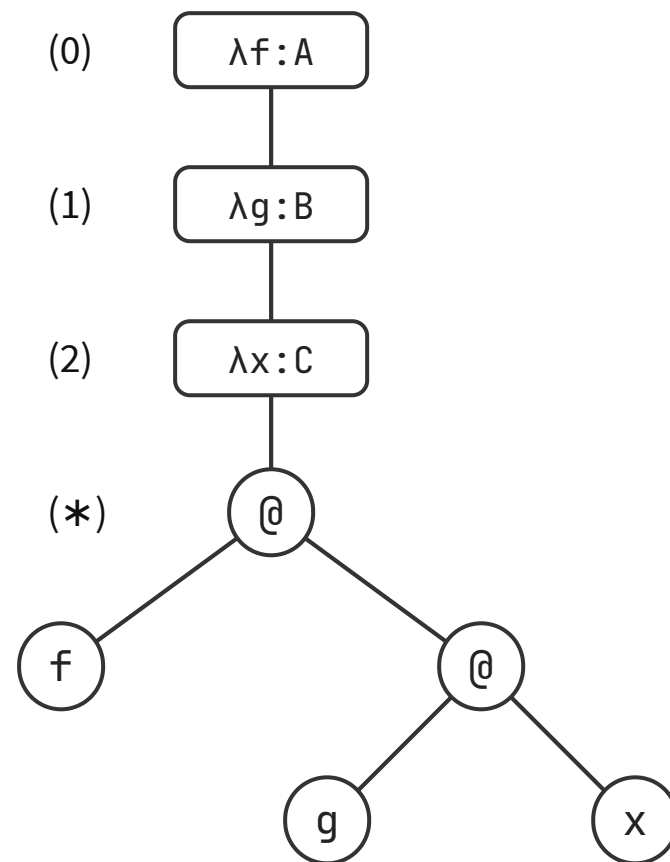
2. a^{int} , c^{int} , i^{int} , r^{long}

3. a^{int} , c^{int} , i^{char}

+ Глобальные определения: $<$, $+$, $*$... — это наши константы

Контекст типизации: пример на лямбда-исчислении

В чистом лямбда-исчислении имена вводятся с помощью абстракции: $\lambda f^A g^B x^C. f (g x)$ по Чёрчу.



Контекст типизации для каждой вершины:

(0) пустой

(1) f^A

(2) f^A, g^B

(*) f^A, g^B, x^C — и во всех вершинах ниже

Правила типизации $\lambda_{\rightarrow}$

- **Δ Синтаксис типов** $T ::= \alpha \mid T \rightarrow T$ — буквы и стрелка, стрелка группируется вправо; наше расширение — ещё константы `Int` и `Bool`. Произвольный тип обозначаем A, B, C
- **Δ Утверждение типизации** $\Gamma \vdash M : A$ — «в контексте Γ терм M имеет тип A »
- **Δ Контекст Γ** — объявления x^A через запятую, переменные различны

 **Устно:** Вспомните правила с лекции: по одному на каждый вид терма.

Правила с лекции

$$\frac{x^A \in \Gamma}{\Gamma \vdash x : A} \text{ (акс)}$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} (\rightarrow E)$$

$$\frac{\Gamma, x^A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} (\rightarrow I)$$

- По Чёрчу отличается только запись абстракции в $(\rightarrow I)$: $\lambda x^A. M$ вместо $\lambda x. M$

Правила для констант + наше расширение

Константы — тоже вершины дерева, и у каждого вида своё правило: литералы и **true/false** — листья, операторы и **if** — вершины с поддеревьями.

Натуральные числа

$$\frac{n \in \{0, 1, 2, \dots\}}{\Gamma \vdash n : \text{Int}} \text{ (нат)}$$

$$\frac{\Gamma \vdash M : \text{Int} \quad \Gamma \vdash N : \text{Int}}{\Gamma \vdash M + N : \text{Int}} (+)$$

Були

$$\frac{}{\Gamma \vdash \text{true} : \text{Bool}} \text{ (бул)}$$

$$\frac{}{\Gamma \vdash \text{false} : \text{Bool}} \text{ (бул)}$$

$$\frac{\Gamma \vdash M : \text{Int} \quad \Gamma \vdash N : \text{Int}}{\Gamma \vdash M < N : \text{Bool}} (<)$$

$$\frac{\Gamma \vdash C : \text{Bool} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : A}{\Gamma \vdash \text{if } C \ M \ N : A} (\text{if})$$

- Правила для $-$, $*$ — как $(+)$; для $==$ — как $(<)$

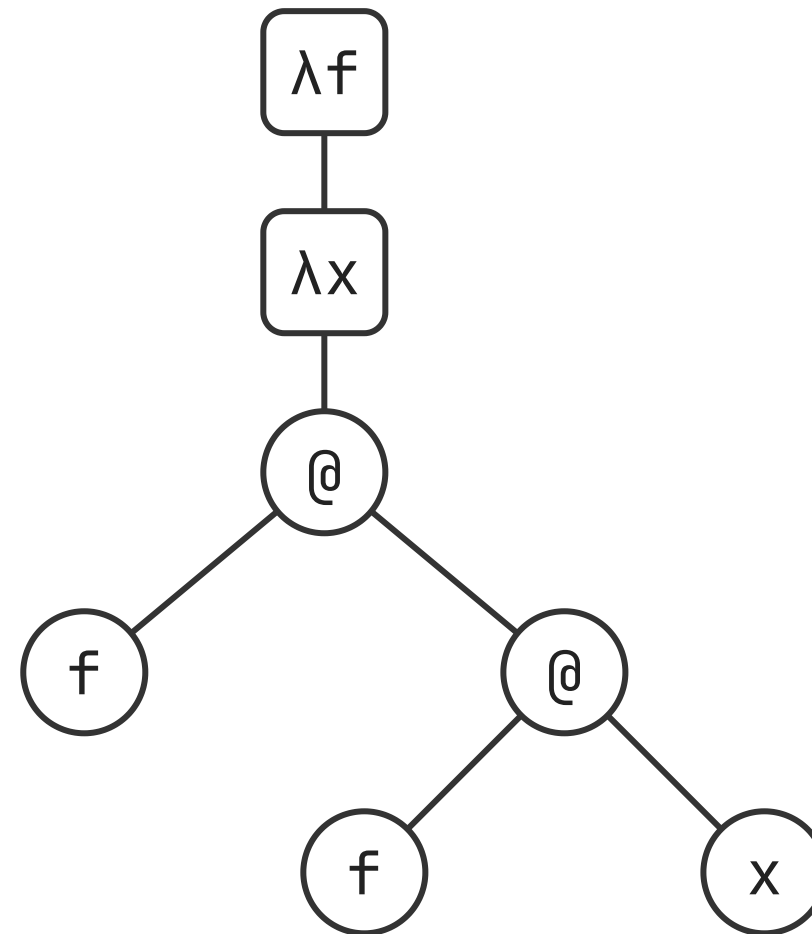
Проверка типа: от дерева терма к дереву вывода

Проверим догадку формально: $\vdash \lambda f x. f (f x) : (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$. Начнём с дерева терма.

$$\frac{x^A \in \Gamma}{\Gamma \vdash x : A} \text{ (акс)}$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} (\rightarrow E)$$

$$\frac{\Gamma, x^A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} (\rightarrow I)$$



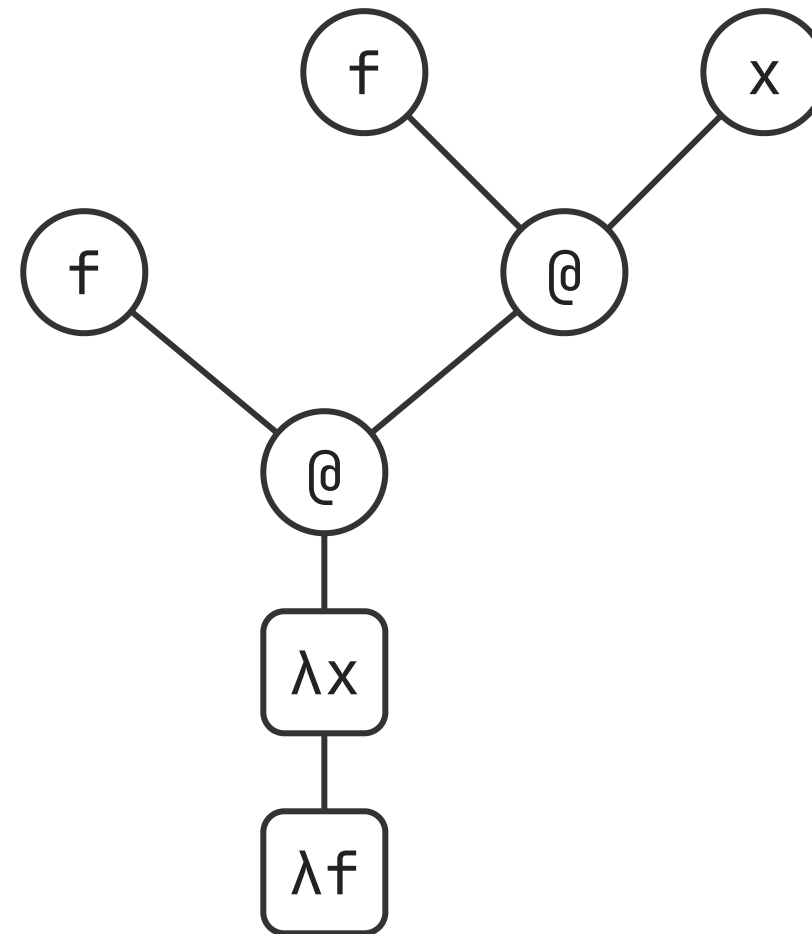
Проверка типа: от дерева терма к дереву вывода

Перевернём: корень внизу, листья наверху — так растут деревья вывода.

$$\frac{x^A \in \Gamma}{\Gamma \vdash x : A} \text{ (акс)}$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} (\rightarrow E)$$

$$\frac{\Gamma, x^A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} (\rightarrow I)$$



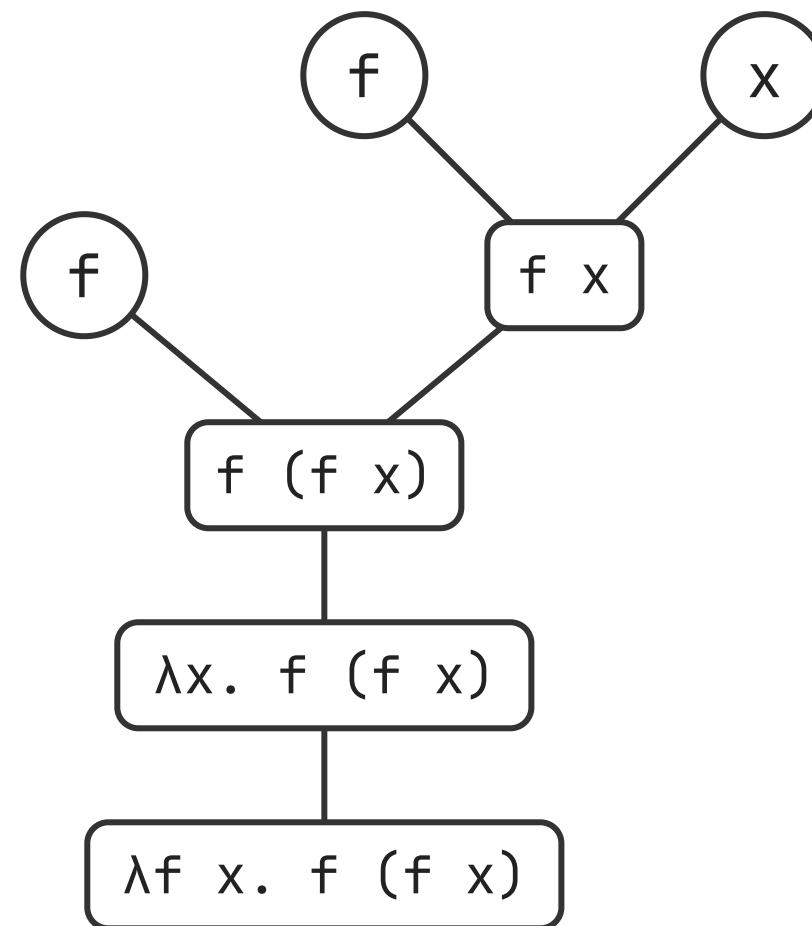
Проверка типа: от дерева терма к дереву вывода

Подпишем в каждой вершине её подтерм целиком: именно про него будет говорить утверждение типизации в этой вершине.

$$\frac{x^A \in \Gamma}{\Gamma \vdash x : A} \text{ (акс)}$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} (\rightarrow E)$$

$$\frac{\Gamma, x^A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} (\rightarrow I)$$



Проверка типа: от дерева терма к дереву вывода

Применяем правила снизу вверх. Выбора нет: вид подтерма — абстракция, аппликация или переменная — сам называет правило. Здесь $\Gamma \equiv f^{\alpha \rightarrow \alpha}, x^{\alpha}$.

$$\frac{x^A \in \Gamma}{\Gamma \vdash x : A} \text{ (акс)}$$

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash M N : B} (\rightarrow E)$$

$$\frac{\Gamma, x^A \vdash M : B}{\Gamma \vdash \lambda x. M : A \rightarrow B} (\rightarrow I)$$

$$\frac{\frac{\frac{\frac{\frac{\Gamma \vdash f : \alpha \rightarrow \alpha}{\Gamma \vdash f : \alpha \rightarrow \alpha} \text{ (акс)}}{\Gamma \vdash f : \alpha \rightarrow \alpha} \text{ (акс)}}{\Gamma \vdash f : \alpha \rightarrow \alpha} \text{ (акс)}}{\Gamma \vdash f : \alpha \rightarrow \alpha} \text{ (акс)} \quad \frac{\frac{\frac{\Gamma \vdash x : \alpha}{\Gamma \vdash x : \alpha} \text{ (акс)}}{\Gamma \vdash x : \alpha} \text{ (акс)}}{\Gamma \vdash x : \alpha} \text{ (акс)} \quad \frac{\Gamma \vdash f : \alpha \rightarrow \alpha \quad \Gamma \vdash x : \alpha}{\Gamma \vdash f x : \alpha} (\rightarrow E)}{\Gamma \vdash f x : \alpha} (\rightarrow E) \quad \frac{\Gamma \vdash f x : \alpha}{f^{\alpha \rightarrow \alpha}, x^{\alpha} \vdash f (f x) : \alpha} (\rightarrow I) \quad \frac{f^{\alpha \rightarrow \alpha}, x^{\alpha} \vdash f (f x) : \alpha}{f^{\alpha \rightarrow \alpha} \vdash \lambda x. f (f x) : \alpha \rightarrow \alpha} (\rightarrow I) \quad \frac{f^{\alpha \rightarrow \alpha} \vdash \lambda x. f (f x) : \alpha \rightarrow \alpha}{\vdash \lambda f x. f (f x) : (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha} (\rightarrow I)$$

ШАГИ

1. λf — абстракция: $(\rightarrow I)$, $f^{\alpha \rightarrow \alpha}$ уходит в контекст
2. λx — абстракция: $(\rightarrow I)$, в контекст уходит x^{α}
3. $f (f x)$ — аппликация: $(\rightarrow E)$, две ветки
4. f — переменная: (акс), $f^{\alpha \rightarrow \alpha} \in \Gamma$
5. $f x$ — аппликация: $(\rightarrow E)$, снова две ветки
6. f — переменная: (акс)
7. x — переменная: (акс), $x^{\alpha} \in \Gamma$ — все листья закрыты

Утверждение проверено: дерево вывода — это перевёрнутое дерево терма, в котором каждая вершина снабжена контекстом и типом.

Дерево вывода с дырками

✎ Заполните пронумерованные пропуски — типы и названия правил — в дереве вывода для $\vdash \lambda x. \text{if } (x < 2) (x + 1) 0 : \text{Int} \rightarrow \text{Int}$.

$$\frac{\frac{\frac{}{\Gamma \vdash x : \Box_5} \text{(акс)}}{\Gamma \vdash x < 2 : \Box_3} \text{(<)} \quad \frac{\frac{}{\Gamma \vdash 2 : \text{Int}} \Box_4 \quad \frac{\frac{}{\Gamma \vdash x : \text{Int}} \Box_7 \quad \frac{}{\Gamma \vdash 1 : \text{Int}} \text{(нат)}}{\Gamma \vdash x + 1 : \text{Int}} \Box_6 \quad \frac{}{\Gamma \vdash 0 : \Box_8} \text{(нат)}}{x^{\Box_1} \vdash \text{if } (x < 2) (x + 1) 0 : \text{Int}} \Box_2$$
$$\vdash \lambda x. \text{if } (x < 2) (x + 1) 0 : \text{Int} \rightarrow \text{Int}$$

Здесь $\Gamma \equiv x^{\text{Int}}$.

✂ Решение.

$\Box_1 = \text{Int}$ (контекст после $(\rightarrow I)$); $\Box_2 = (\text{if})$; $\Box_3 = \text{Bool}$; $\Box_4 = (\text{нат})$; $\Box_5 = \text{Int}$; $\Box_6 = (+)$; $\Box_7 = (\text{акс})$; $\Box_8 = \text{Int}$ — ветки **if** одного типа.

Дерево вывода с нуля

✎ Постройте дерево вывода для $\vdash \lambda x f. f x : \alpha \rightarrow (\alpha \rightarrow \beta) \rightarrow \beta$

✚ **Решение.**

Пусть $\Gamma \equiv x^\alpha, f^{\alpha \rightarrow \beta}$.

$$\frac{\frac{\frac{\overline{\Gamma \vdash f : \alpha \rightarrow \beta} \text{ (акс)}}{x^\alpha, f^{\alpha \rightarrow \beta} \vdash f x : \beta} \text{ (}\rightarrow E\text{)}}{x^\alpha \vdash \lambda f. f x : (\alpha \rightarrow \beta) \rightarrow \beta} \text{ (}\rightarrow I\text{)}}{\vdash \lambda x f. f x : \alpha \rightarrow (\alpha \rightarrow \beta) \rightarrow \beta} \text{ (}\rightarrow I\text{)}$$

Верно ли утверждение?

✎ **Устно:** Поднимите руку, если утверждение выводимо:

(a) $\vdash \mathbf{K} : \alpha \rightarrow \alpha \rightarrow \alpha$

(б) $\vdash \mathbf{K} : \alpha \rightarrow \alpha \rightarrow \beta$

(в) $a^{\alpha \rightarrow \alpha} \vdash \mathbf{I} a : \alpha \rightarrow \alpha$

(г) $a^{\alpha \rightarrow \alpha} \vdash a b : \alpha$

(д) $\vdash \lambda x. x : \text{Int} \rightarrow \text{Int}$

(е) $\vdash \lambda x. x + 1 : \alpha \rightarrow \alpha$

✂ Решение.

Выводимы (а), (в), (д). В (а) тип не наиболее общий, но дерево строится. В (б) результат — первый аргумент, он типа α . В (г) переменной b нет в контексте. (д) и (е) — снова буквы против констант: вместо буквы Int подставить можно, а сделать из Int букву — нет.

Простые примеры задач населения

✎ Найдите все термы M : $x^\beta \vdash M : \alpha \rightarrow \alpha$

✎ Найдите все термы M : $x^\beta \vdash M : \alpha \rightarrow \beta$

✎ Найдите все термы M : $x^\beta \vdash M : \alpha \rightarrow \gamma$

✚ Решение.

- Единственный терм типа α передаётся аргументом — обязаны его вернуть: $M \equiv \lambda y. y$
- Единственное значение типа β есть в контексте — обязаны вернуть его: $M \equiv \lambda y. x$
- Значение типа γ брать неоткуда: этот тип не населён в данном контексте

Сколько обитателей у типа?

✎ Сколько замкнутых термов в нормальной форме у каждого типа?

(а) $\alpha \rightarrow \alpha \rightarrow \alpha$

(б) $(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$

(в) α

(г) Int

⚡ Решение.

- (а) Два: $\lambda x y. x$ и $\lambda x y. y$ — больше взять значение типа α неоткуда
- (б) Бесконечно много: $\lambda s z. z$, $\lambda s z. s z$, $\lambda s z. s (s z)$, ... — это числа Чёрча!
- (в) Ни одного: у буквы нет своих значений, а аргументов, из которых её можно взять, нет
- (г) Бесконечно много: $0, 1, 2, \dots$ — у константного типа значения свои

Рекурсия возвращается: **fix** факультативно

- $Y := \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$ содержит $x x$ — в $\lambda_{\rightarrow}$ у него нет типа, как и у любого другого комбинатора неподвижной точки: все программы $\lambda_{\rightarrow}$ завершаются
- Значит, рекурсию встраиваем в язык так же, как числа и були: новая форма терма **fix** M , от которой нужно ровно свойство **Y** из П2 — **fix** $M =_{\beta} M (\text{fix } M)$

 **Факультативно:** Задайте **fix** полностью: (а) строку в грамматике термов; (б) правило типизации; (в) правило редукции.

✚ Решение.

$$(a) \ M ::= \dots \mid \text{fix } M \quad (б) \ \frac{\Gamma \vdash M : A \rightarrow A}{\Gamma \vdash \text{fix } M : A} \text{ (fix)} \quad (в) \ \text{fix } M \rightsquigarrow M (\text{fix } M)$$

Аргумент **fix** — функция от «самой себя»: тип $A \rightarrow A$; результат — неподвижная точка, тип A .

Факториал в расширенном языке

факультативно

$$\mathbf{fac} := \mathbf{fix} \ F, \quad F \equiv \lambda f \ n. \mathbf{if} \ (n == 0) \ 1 \ (n * f \ (n - 1))$$

- Как в П2: рекурсивный вызов абстрагирован по имени f — только вместо $\mathbf{Y}$ стоит $\mathbf{fix}$, а вместо кодировок — константы

 **Устно:** Какой тип у F ? у $\mathbf{fac}$?

↗ Решение.

$F : (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int} \rightarrow \text{Int}$ — принимает «себя» и число; по правилу $(\mathbf{fix})$ с $A := \text{Int} \rightarrow \text{Int}$ получаем $\mathbf{fac} : \text{Int} \rightarrow \text{Int}$.

Дерево типизации факториала факультативно

$F \equiv \lambda f n. \text{if } (n == 0) 1 (n * f (n - 1)), \quad \Gamma \equiv f^{\text{Int} \rightarrow \text{Int}}, n^{\text{Int}}$

$$\begin{array}{c}
 \frac{\frac{\frac{}{} \text{акс}}{\Gamma \vdash n : \text{Int}} \quad \frac{\frac{}{} \text{нат}}{\Gamma \vdash 0 : \text{Int}}}{\Gamma \vdash n == 0 : \text{Bool}} \text{ (==)} \quad \frac{\frac{}{} \text{нат}}{\Gamma \vdash 1 : \text{Int}} \quad \frac{\frac{\frac{}{} \text{акс}}{\Gamma \vdash n : \text{Int}} \quad \frac{\frac{\frac{\frac{}{} \text{акс}}{\Gamma \vdash f : \text{Int} \rightarrow \text{Int}} \quad \frac{\frac{\frac{\frac{}{} \text{акс}}{\Gamma \vdash n : \text{Int}} \quad \frac{\frac{}{} \text{нат}}{\Gamma \vdash 1 : \text{Int}}}{\Gamma \vdash 1 : \text{Int}} \text{ (nat)}}{\Gamma \vdash n - 1 : \text{Int}} \text{ (-)}}{\Gamma \vdash n - 1 : \text{Int}} \text{ (-)}}{\Gamma \vdash f (n - 1) : \text{Int}} \text{ (→E)}}{\Gamma \vdash n * f (n - 1) : \text{Int}} \text{ (*)}}{\Gamma \vdash n * f (n - 1) : \text{Int}} \text{ (if)}}{\Gamma \vdash \text{if } (n == 0) 1 (n * f (n - 1)) : \text{Int}} \text{ (→I)}}{\frac{f^{\text{Int} \rightarrow \text{Int}} \vdash \lambda n. \text{if } \dots : \text{Int} \rightarrow \text{Int}}{\vdash F : (\text{Int} \rightarrow \text{Int}) \rightarrow \text{Int} \rightarrow \text{Int}} \text{ (→I)}}{\vdash \text{fix } F : \text{Int} \rightarrow \text{Int}} \text{ (fix)}
 \end{array}$$

Как работает fix: fac 2

факультативно

Раскрыли **fac**; дальше единственное правило для **fix** — редукция $\mathbf{fix} \ M \rightsquigarrow M \ (\mathbf{fix} \ M)$.

fix **F** 2

ЦЕПОЧКА

fac 2 $\rightsquigarrow_{\mathbf{fac}}$ **fix** **F** 2

Как работает `fix`: `fac 2` факультативно

Правило (`fix`): наружу вышла копия F , внутри остался `fix` F — запас на следующий виток.

`F` (`fix` `F`) `2`

ЦЕПОЧКА

`fac 2` $\rightsquigarrow_{\text{fac}}$ `fix` F `2`

$\rightsquigarrow_{\text{fix}}$ F (`fix` F) `2`

Как работает fix: fac 2 факультативно

Два β -шага: f стало `fix F`, n стало 2. Перед нами тело F — обычный `if` с константами.

```
if (2 == 0) 1 (2 * fix F (2 - 1))
```

ЦЕПОЧКА

`fac 2` $\rightsquigarrow_{\text{fac}}$ `fix F 2`

$\rightsquigarrow_{\text{fix}}$ `F (fix F) 2`

$\rightarrow_{\beta}$ `if (2 == 0) 1 (2 * fix F (2 - 1))`

Как работает fix: fac 2 факультативно

Константы считаются за шаг: $2 == 0 \rightsquigarrow \text{false}$, if выбрал ветку else , $2 - 1 \rightsquigarrow 1$. Снаружи остался $2 *$, внутри — тот же кадр, что в начале.

$2 * \text{fix } F \ 1$

ЦЕПОЧКА

$\text{fac } 2 \rightsquigarrow_{\text{fac}} \text{fix } F \ 2$

$\rightsquigarrow_{\text{fix}} F (\text{fix } F) \ 2$

$\rightarrow_{\beta} \text{if } (2 == 0) \ 1 \ (2 * \text{fix } F \ (2 - 1))$

$\rightarrow 2 * \text{fix } F \ 1$ — правила $(==)$, (if) , $(-)$

Как работает fix: fac 2 факультативно

Второй виток: снова (`fix`), β -шаги и условие; снаружи ещё один множитель.

$2 * 1 * \text{fix } F \ 0$

ЦЕПОЧКА

$\text{fac } 2 \rightsquigarrow_{\text{fac}} \text{fix } F \ 2$

$\rightsquigarrow_{\text{fix}} F (\text{fix } F) \ 2$

$\rightarrow_{\beta} \text{if } (2 == 0) \ 1 \ (2 * \text{fix } F \ (2 - 1))$

$\rightarrow 2 * \text{fix } F \ 1$ — правила $(==)$, (if) , $(-)$

$\rightarrow 2 * 1 * \text{fix } F \ 0$ — **ВИТОК 2**

Как работает fix: fac 2 факультативно

Третий виток: копия, β -шаги — и на этот раз $0 == 0$ истинно.

$2 * 1 * \text{if } (0 == 0) 1 (0 * \text{fix } F (0 - 1))$

ЦЕПОЧКА

$\text{fac } 2 \rightsquigarrow_{\text{fac}} \text{fix } F 2$

$\rightsquigarrow_{\text{fix}} F (\text{fix } F) 2$

$\rightarrow_{\beta} \text{if } (2 == 0) 1 (2 * \text{fix } F (2 - 1))$

$\rightarrow 2 * \text{fix } F 1$ — правила $(==)$, (if) , $(-)$

$\rightarrow 2 * 1 * \text{fix } F 0$ — ВИТОК 2

$\rightarrow 2 * 1 * \text{if } (0 == 0) 1 (0 * \text{fix } F (0 - 1))$ —
ВИТОК 3

Как работает fix: fac 2 факультативно

`if` вернул 1, ветка с `fix F (0 — 1)` выброшена, так и не раскрывшись: `fix` разворачивался ровно столько раз, сколько потребовало условие.

$$2 * 1 * 1 \Rightarrow 2$$

ЦЕПОЧКА

$\text{fac } 2 \rightsquigarrow_{\text{fac}} \text{fix } F \ 2$

$\rightsquigarrow_{\text{fix}} F (\text{fix } F) \ 2$

$\rightarrow_{\beta} \text{if } (2 == 0) \ 1 \ (2 * \text{fix } F \ (2 - 1))$

$\rightarrow 2 * \text{fix } F \ 1$ — правила $(==)$, (if) , $(-)$

$\rightarrow 2 * 1 * \text{fix } F \ 0$ — виток 2

$\rightarrow 2 * 1 * \text{if } (0 == 0) \ 1 \ (0 * \text{fix } F \ (0 - 1))$ —
ВИТОК 3

$\rightarrow 2 * 1 * 1 \Rightarrow 2$ — правила $(==)$, (if) , $(*)$

Сегодня мы многое поняли

- Анализ типов — по правилам расставляем метки — просто!
- Научили мозг проверке типов — отличать хорошие программы от плохих
- Объяснить по всей строгости, почему программа типизируется или нет

Таинственные связи ☁

Дерево вывода типа комбинатора **B** для произвольных типов A, B, C — или вывод силлогизма в исчислении высказываний?

$$\frac{\frac{\frac{\overline{\Gamma \vdash f : B \rightarrow C} \text{ (акс)}}{f^{B \rightarrow C}}, \frac{\frac{\overline{\Gamma \vdash g : A \rightarrow B} \text{ (акс)}}{\Gamma \vdash g x : B} \text{ (}\rightarrow E\text{)}}{\Gamma \vdash g x : B} \text{ (}\rightarrow E\text{)}}{f^{B \rightarrow C}, g^{A \rightarrow B}, x^A \vdash f (g x) : C} \text{ (}\rightarrow E\text{)}}{\frac{f^{B \rightarrow C}, g^{A \rightarrow B} \vdash \lambda x. f (g x) : A \rightarrow C} \text{ (}\rightarrow I\text{)}}{\frac{f^{B \rightarrow C} \vdash \lambda g x. f (g x) : (A \rightarrow B) \rightarrow A \rightarrow C} \text{ (}\rightarrow I\text{)}}{\vdash \lambda f g x. f (g x) : (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C} \text{ (}\rightarrow I\text{)}}$$

Таинственные связи ☁

Дерево вывода типа комбинатора **B** для произвольных типов A, B, C — или вывод силлогизма в исчислении высказываний?

$$\frac{\frac{\frac{\Gamma \vdash B \rightarrow C \text{ (акс)}}{B \rightarrow C}, \frac{\frac{\frac{\Gamma \vdash A \rightarrow B \text{ (акс)}}{A \rightarrow B}, \frac{\frac{\Gamma \vdash A \text{ (акс)}}{A}}{B} \text{ (}\rightarrow E\text{)}}{C} \text{ (}\rightarrow E\text{)}}{B \rightarrow C, A \rightarrow B \vdash A \rightarrow C} \text{ (}\rightarrow I\text{)}}{B \rightarrow C \vdash (A \rightarrow B) \rightarrow A \rightarrow C} \text{ (}\rightarrow I\text{)}}{\vdash (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C} \text{ (}\rightarrow I\text{)}$$

Материалы

Типизация в стиле Карри и в стиле Чёрча

	по Карри	по Чёрчу
абстракция	$\lambda x. M$	$\lambda x^A. M$ — в домашке <code>\x:a. x</code>
количество кода	меньше кода, сложнее задача типовой подсистеме языка	программисту приходится больше писать
сколько типов у терма	либо 0, либо ∞	либо 0, либо 1
алгоритм	сложнее: приходится решать систему типовых уравнений (П13)	проще: вся необходимая информация написана в терме
в вашем языке	Haskell, OCaml; <code>auto</code> в C++, <code>var</code> в Java и C#	типы параметров в C, Java, Go

Мотивация статического анализа программ

- Программное обеспечение — это сложно
- В программах случаются ошибки (постоянно и неизбежно)
- Поиск ошибок может занимать годы, десятилетия, вечность
- Нужны методы для того, чтобы судить о программах и их свойствах автоматически
- Программы могут работать долго
- Хотим отвергать «плохие» программы до исполнения (статически)

Роль типов в языках

- Позволяют выявлять некоторые классы ошибок:

```
GHCi> length 42      -- Ошибка: на вход не список, а число
GHCi> foldr (+) "ABCDE" -- Ошибка: плюс работает на числах, но не символах
```

- Механизм абстракции: интерфейс, скрывающий детали реализации
- Средство документирования: как использовать сущность
- *Частичная* спецификация поведения
- Позволяют генерировать более эффективный низкоуровневый код

Ограниченность систем типов ☁

- Хотелось бы, чтобы все «хорошие» программы типизировались, а все «плохие» — нет
- Однако по теореме Успенского-Райса задача проверки любого нетривиального¹ свойства программы неразрешима

В чем подвох с системами типов?

- Всё, что типизируется, — «хорошее», но обратное не всегда верно
- ➡ Всегда существуют «хорошие» программы, которые не типизируются²



1. Существуют программы, как обладающие таким свойством, так и не обладающие.

2. На практике к тому же зачастую существуют «плохие», которые типизируются.

Деревья вывода

- Теория доказательств математической логики — доказываем принадлежность утверждения типизации множеству истинных утверждений
- Истинное утверждение в теории доказательств — то, которое можно *синтаксически* вывести по правилам вывода из аксиом
- Деревья вывода — синтаксические, машинно-проверяемые конструкции
- Деревья сами по себе не несут смысла, наличие корректного дерева говорит только о возможности его построения
- Чтобы деревья имели смысл, для каждой теории доказываются утверждения вида:
 - Если существует дерево вывода с корнем X , то верно Y
 - **e.g.** Если существует $\lambda_{\rightarrow}$ дерево с утверждением про терм M в корне (терм M в таком-то контексте имеет такой-то тип), то этот терм строго нормализуем
- Иногда по правилам вывода очевиден алгоритм построения дерева
 - В нашем случае (для $\lambda_{\rightarrow}$) это так
 - В общем случае — нет (математики оказались бы не нужны)

Не всё сильно-нормализуемое типизируется *

Вспомним: всё, что типизируется, «хорошее», но обратное не всегда верно. Будем «хорошим» считать сильную нормализуемость.

2 **K** терм сильно нормализуем и редуцируется так:

$$\mathbf{2} \mathbf{K} = \lambda z. \mathbf{K} (\mathbf{K} z) = \lambda z a b. z$$

Однако протипизировать это не получится.

- Наиболее общий тип **K** — это $\gamma \rightarrow \tau \rightarrow \gamma$
- $\mathbf{2} := \lambda s z. s (s z)$, наиболее общий тип **2** — это $(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$
- По правилу типизации аппликации, применение **2** к **K** накладывает на γ, τ, α ограничения $\alpha = \gamma, \alpha = \tau \rightarrow \gamma$

Противоречие.

Эквивалентности не сохраняют тип *

β -эквивалентность не сохраняет тип

Из лекции известно, что если M редуцируется к N и $\Gamma \vdash M : \tau$, то $\Gamma \vdash N : \tau$.

Неправда, что если M редуцируется к N и $\Gamma \vdash N : \tau$, то $\Gamma \vdash M : \tau$.

Пример: $N = \lambda x. x$, $N : \alpha \rightarrow \alpha$; $M = \lambda x. (\lambda y. x) (x \mathbf{I})$, и за одно редуцирование M переходит в N , но $M : \alpha \rightarrow \alpha$ не выполняется: тогда бы подтерм $x \mathbf{I}$ не типизировался, так как α не является стрелочным типом.

η -эквивалентность не сохраняет тип

Если $\lambda x. M x : \tau$ (где x не является свободным в M), то $M : \tau$.

Если $M : \tau$, то не обязательно $\lambda x. M x : \tau$.

Пример: $\mathbf{I} : \alpha \rightarrow \alpha$, но $\lambda x y. x y : (\beta \rightarrow \gamma) \rightarrow (\beta \rightarrow \gamma)$ нельзя приписать тип $\alpha \rightarrow \alpha$: тогда бы подтерм $x y$ не типизировался, так как α не является стрелочным типом.

Что посмотреть в транспорте

- Формальная верификация операционных систем

Серьёзные материалы

- [Соответствие Карри–Говарда: от матлогики к программированию](#)
- [Вывод типов от Хиндли-Милнера до GHC 8.8](#)
- [Программирование с зависимыми типами на языке Idris](#)
- [Edwin Brady «Type-Driven Development with Idris»](#)
- Pierce, Benjamin C. Types and programming languages. MIT press, 2002.
- Пирс Б. Типы в языках программирования. — Лямбда пресс, Добросвет, 2012. — 656 с.
- [Sandy Maguire «Thinking with Types. Type-Level Programming in Haskell»](#)
- Henk Barendregt, Wil Dekkers, and Richard Statman. «Lambda Calculus with Types». Cambridge University Press, New York, NY, USA, 2013.
- Dunfield, Jana, and Neel Krishnaswami. «Bidirectional typing.» *ACM Computing Surveys (CSUR)* 54.5 (2021): 1-38.